Relatório: Os Pilares da Programação Orientada a Objetos (POO)

Conceitos fundamentais

Classe: Modelo ou planta que define atributos (dados) e métodos (comportamentos) de um conjunto de objetos. É o “molde”.
Objeto: Instância concreta de uma classe, criada em tempo de execução. É o “produto” obtido a partir do molde.
Atributo: Variável que representa uma característica ou estado de um objeto. Exemplo: nome, idade.
Método: Função definida dentro de uma classe que descreve o comportamento do objeto. Exemplo: calcularIdade().
Construtor: Método especial chamado automaticamente ao criar um objeto, usado para inicializar seus atributos.
Instância: Cada objeto criado a partir da mesma classe possui seus próprios dados, mas compartilha a estrutura.

Os quatro pilares

1. Encapsulamento

Agrupa dados e comportamentos em uma única unidade (a classe) e controla o acesso a eles. Oculta a implementação interna e expõe apenas o que é necessário, protegendo o estado do objeto contra alterações indevidas.

- Modificadores de acesso: public (acesso livre), protected (acesso pela própria classe e herdeiros), private (acesso apenas pela própria classe).
- Exemplo: atributos privados acessados por getters (leitura) e setters (escrita) com validações.

2. Herança

Permite que uma classe (subclasse/filha) herde atributos e métodos de outra classe (superclasse/pai), formando uma hierarquia. Promove o reaproveitamento de código e evita repetição.

- Tipos: simples (uma classe pai), múltipla (várias classes pai) e hierárquica (cadeia de herança).
- Relação “é um”: um Gato é um Animal.
- Exemplo: a classe Gato herda de Animal e reaproveita o atributo nome.

3. Polimorfismo

Capacidade de objetos de diferentes classes responderem ao mesmo método de formas distintas. Permite tratar objetos de forma genérica, simplificando o código.

- Tipos: por sobrescrita (redefinição do método na subclasse) e por sobrecarga (mesmo nome com parâmetros diferentes).

- Exemplo: som() de Cachorro faz “au au” e o de Gato faz “miau”.

4. Abstração

Mostra apenas os aspectos essenciais de um objeto, escondendo os detalhes internos complexos. O usuário interage com o “quê” fazer, não com o “cómo” funciona.

- Classes abstratas: não podem ser instanciadas e podem exigir métodos que as subclasses devem implementar.
- Interfaces: contrato que define quais métodos uma classe deve ter.
- Exemplo: ligar o carro sem conhecer o funcionamento do motor.